

	Flow & Recoverability	Verification	Context	Product Integrity	Control & Accountability
PILLARS → LEVELS ↓	A change reaches production, and can be undone, quickly and safely.	A change is known to be correct by someone who didn't write it.	The system is understandable to whoever — or whatever — works on it next.	What ships is deliberate, wanted, and coherent with what is already there.	Every change has a named human owner and a bounded blast radius.
3 Self-correcting	Canary thresholds and rollout speed adjust from the observed failure rate for that class of change. Fault injection runs continuously; a regression in restore time raises work before an incident does. The path to production tightens or widens per change class without anyone running a project to do it.	An escaped defect closes with the missing check added; the gate that failed is tightened as part of the fix rather than as follow-up work. Flaky tests are quarantined and repaired automatically, so signal quality does not depend on human triage. Check strictness and eval thresholds move with the measured risk of each change class.	Context is regenerated from the system as it changes; documents failing their checks are corrected and re-proposed automatically. Repeated agent failure in the same area raises a context fix rather than a retry. Instructions are pruned on evidence of being unread, unused or contradictory, so the corpus doesn't bloat.	Work that fails its hypothesis enters the retirement path on a timer unless someone argues to keep it — the default is removal, not survival. The bar for entering the backlog moves with the measured hit rate. Adding to the surface forces an explicit decision about what leaves.	Permissions narrow themselves to observed need; unused grants expire without anyone revoking them. Sandbox boundaries and approval thresholds adjust to each agent's demonstrated failure rate on that class of work. Spend limits recalibrate against delivered value rather than being reset by argument.
2 Instrumented	Time to restore is measured per class of change, not as a team-wide average. The rollback mechanism is exercised on a schedule and its success rate is known. The fraction of traffic exposed before a bad change is stopped is measured rather than assumed.	Every escaped defect is traced to the specific check that should have caught it, and the gap is recorded. Detection power is tracked over time, and the suite's false-positive rate is measured and budgeted — flakiness has a number and an owner. First-pass acceptance of agent-authored changes is measured, and falling acceptance is treated as a signal about the constraints, not about the agent.	The gap between system and documentation is measured — staleness, and time-to-first-useful-change for a newcomer or a fresh agent session. Agent failures attributable to missing or wrong context are counted and traced to the specific document. Duplication and coupling are trended; cognitive debt has a number that someone reviews.	The proportion of shipped work that met its stated hypothesis is known, and quoted. Every feature's usage is measured against its cost to keep. The size of the product surface is a tracked number with a direction of travel.	Permission grants are measured against use: what agents actually reached versus what they were permitted to. Provenance completeness is measured, and unattributed changes are counted and chased. Inference spend is measured per unit of delivered work, and blast radius is quantified per change class.
1 Constrained	The automated path is the only path to production; there is no manual route around it. Changes ship decoupled from release — dark, behind flags, or progressively rolled out — so a bad change reaches a fraction of traffic before it is stopped. Rollback is triggered by health signals rather than by a human noticing.	Machine-checkable constraints — types, contracts, lint, security scans, performance budgets — run before a human sees the change, and agents iterate against them unattended. The suite's detection power is evidenced by mutation score or deliberate fault injection, not inferred from coverage. Behaviour that cannot be asserted deterministically is gated by evals with agreed thresholds.	Documentation and instruction files are validated by the pipeline: stale commands, broken references and contradictory rules fail the build. The context agents need is retrievable through a defined interface rather than scattered across tools. Changes that duplicate existing code or widen coupling beyond a threshold are stopped by the system, not by a reviewer's memory.	Work cannot enter the backlog without a stated hypothesis and a named owner; the system will not accept it otherwise. Every launch carries a review date, and the outcome is recorded whether or not it flatters the decision. Retirement is a funded routine with an owner; feature flags carry expiry dates and are removed on schedule.	Agents are sandboxed by default; access to data, tools and networks is granted per task and expires. Provenance — model, instruction version, accountable human — is recorded automatically and cannot be omitted. Irreversible operations require a human decision by construction, and spend limits halt a runaway loop rather than surfacing on the bill.
0 Owned	Deployment is automated and any engineer on the team can run it unaided. Rollback is a documented procedure with a named owner, deliberately exercised at least once. Everything needed to rebuild the system is in version control, and environments are provisioned from it.	Every change is reviewed by a named engineer who is accountable for it having merged. The suite runs on every change and a red build blocks the merge. Agent-authored changes are held to the same review standard as human-authored ones.	Every component has a named owner and current documentation of its purpose and how to run it. Agent instructions are maintained deliberately and reviewed like code. Architectural decisions are recorded with the rationale that applied at the time.	Every item of work has a named owner and a stated intended outcome, agreed before build starts. Someone holds the authority to decline work, and exercises it. The team can say what it declined last quarter, and why.	Every change has a named human accountable for it, whether or not they wrote it. Agents run under scoped credentials, with a documented list of what they may reach. Agent authorship is attributed in the history; inference spend is attributed to teams and reviewed.
-1 Unowned	Deployment is a scheduled event, performed by hand, by whoever is available. Rollback is a rebuild and redeploy of unknown duration; it has not been exercised this quarter. Environments differ from production in ways nobody can enumerate.	Failing tests are muted, skipped, or re-run until green. Approvals arrive faster than the diff could plausibly have been read. Nobody can say what the suite proves; coverage is quoted instead.	Documentation exists and is known to be wrong; people ask a specific individual instead. Agent instruction files have accumulated contradictory rules that nobody has pruned. Changes are made by pattern-matching nearby code; no one can explain why the system is shaped as it is.	Features ship with no stated purpose and no nameable owner. Nothing has ever been retired; the product surface grows monotonically. Requests become work items without anyone testing whether they are wanted.	Agents run with the permissions of whoever launched them; nobody can enumerate what they can reach. Agent-authored changes are indistinguishable from human ones after the fact. Nobody knows what is being spent on inference, or by whom.

In an AI-Native software organisation, agentic delegation is the default mode of production, not an accelerator applied to a human-authored baseline.

When production is cheap, the scarce resource is assurance.